Documento dei Requisiti

Sistema informativo per l'analisi comparativa e l'ottimizzazione di deck in Clash Royale

versione 1.0 del 04/04/2026

Autore

Param Singh Tamna

Indice del Documento

1 Introduzione **1**
1.1 Obiettivi del progetto 1
1.2 Contesto di Business 1
1.3 Attori coinvolti 1
1.4 Idee per le soluzioni 2
1.5 Sommario del documento 2

2 Servizi del sistema **2**
2.1 Analisi del contesto 2
2.2 Requisiti funzionali 3
2.3 Requisiti informativi 4

3 Requisiti non funzionali e vincoli del sistema **4**
3.1 Interfaccia utente, fattori umani e Ambiente fisico 4
3.2 Documentazione 4
3.3 Requisiti hardware e software 5
3.4 Gestione degli errori e delle condizioni al limite 5
3.5 Interfaccia di sistema 5
3.6 Prestazioni e qualità 5
3.7 Manutenibilità 5

4 Modello del sistema **6**
4.1 Elenco dei Casi d'uso 6
4.2 Diagramma dei Casi d'uso 6

5 Analisi del Caso d'uso "Calcolo Matchup" **6**
5.1 Scenario 6
5.2 Nome e descrizione 6
5.3 Attori 6
5.4 Pre-condizioni 6
5.5 Flusso principale degli eventi 6
5.6 Flussi alternativi 7
5.7 Condizioni di errore 7
5.8 Post-condizioni 7
5.9 Priorità e frequenza d'uso 7
5.10 Requisiti speciali 7
5.11 Diagramma del Caso d'uso 7
5.12 Diagramma delle Classi 8
5.13 Diagramma di Attività 8
5.14 Diagramma di Sequenza 9
5.15 Diagramma di Stato 9

6 Appendice **9**
6.1 Glossario 9
6.2 Documentazione 9
6.3 Riferimenti 9

1. INTRODUZIONE

1.1 Obiettivi del progetto

L'obiettivo del progetto è sviluppare un'applicazione software in linguaggio Java in grado di analizzare e confrontare due mazzi di carte del videogioco Clash Royale.

Il sistema permette all'utente di inserire due mazzi composti da 8 carte ciascuno e di ottenere un'analisi del confronto tra i due deck. L'analisi considera diversi fattori, tra cui:

• costo medio di elisir
• presenza di carte evoluzione
• presenza di carte eroe
• tipologia delle carte
Il risultato finale è una percentuale stimata di vittoria del mazzo dell'utente rispetto a quello avversario.

Il software ha inoltre lo scopo di fornire suggerimenti per migliorare la composizione del mazzo.

1.2 Contesto di Business

Negli ultimi anni si è diffuso l'utilizzo di applicazioni di supporto per videogiochi competitivi. Queste applicazioni, chiamate companion app, aiutano i giocatori ad analizzare strategie e prestazioni.

Il progetto prende ispirazione da applicazioni esistenti come DeckAI, che analizzano la composizione dei mazzi nel gioco Clash Royale.

Il sistema sviluppato in questo progetto si differenzia da DeckAI per:

• una interfaccia grafica differente
• maggiore attenzione al calcolo della percentuale di vittoria
• un sistema che evidenzia i counter tra le carte
• suggerimenti automatici per migliorare il deck.

Il progetto ha principalmente scopo didattico, nell'ambito dello studio della programmazione ad oggetti e dell'analisi dei requisiti software.

1.3 Attori coinvolti

Gli attori del sistema sono le entità che interagiscono con il software.

Giocatore (attore principale)
È l'utente che utilizza il software per analizzare i mazzi e ottenere informazioni strategiche prima di una partita.

Sistema di gestione delle carte
È il componente che contiene le informazioni relative alle carte del gioco, come nome, costo in elisir, tipo e caratteristiche speciali.

1.4 Idee per le soluzioni

Il sistema viene sviluppato utilizzando il paradigma della programmazione orientata agli oggetti (OOP).

Le principali entità del sistema vengono rappresentate attraverso classi Java:

• Carta = rappresenta una carta del gioco
• Deck = rappresenta un mazzo composto da 8 carte
• Utente = rappresenta il giocatore che utilizza il software

La logica principale di confronto tra i mazzi è implementata nella classe Deck, attraverso il metodo confrontaDeck().

Questo metodo analizza le proprietà delle carte e calcola una percentuale di vittoria stimata.

1.5 Sommario del documento

Questo documento descrive i requisiti e la struttura del sistema DeckAnalyzer.

In particolare vengono analizzati:

• i servizi forniti dal sistema
• i requisiti funzionali e non funzionali
• il modello del sistema
• i casi d'uso principali
• i diagrammi UML che descrivono il comportamento del software.

2. SERVIZI DEL SISTEMA

2.1 Analisi del contesto

Il software opera come applicazione stand-alone eseguibile su computer tramite Java Virtual Machine (JVM).

L'utente interagisce con il sistema inserendo le carte nei due mazzi da confrontare.

Nel prototipo iniziale l'interazione avviene tramite console, mentre nella versione finale è prevista un'interfaccia grafica.

Il sistema esegue le seguenti operazioni principali:

1. acquisizione delle carte del mazzo dell'utente

2. acquisizione delle carte del mazzo avversario

3. analisi delle caratteristiche dei mazzi

4. calcolo della percentuale di vittoria

5. visualizzazione dei risultati.

2.2 Requisiti funzionali
I requisiti funzionali descrivono le funzionalità che il sistema deve fornire.

RF1 - Inserimento carte
Il sistema deve permettere all'utente di inserire le carte che compongono il proprio mazzo.

RF2 - Creazione del mazzo
Il sistema deve consentire la creazione di un mazzo composto da 8 carte.

RF3 - Gestione delle caratteristiche delle carte
Ogni carta deve contenere le seguenti informazioni:
• nome della carta
• costo in elisir
• tipo di carta
• presenza di evoluzione
• presenza di versione eroe.

RF4 - Confronto tra mazzi
Il sistema deve confrontare due mazzi e analizzare le loro caratteristiche.

RF5 - Calcolo della percentuale di vittoria
Il sistema deve calcolare una percentuale stimata di vittoria del mazzo dell'utente.

RF6 - Visualizzazione dei risultati
Il sistema deve mostrare il risultato dell'analisi all'utente.

ID	Essenziale	Priorità	Stabile	Fonte	Test	Impatto
RF1	SI	ALTA	SI	Giocatore	Test inserimento carte	Se non realizzato l'utente non può creare il mazzo
RF2	SI	ALTA	SI	Regole Clash Royale	Test creazione mazzo	Il sistema non rispetterebbe le regole del gioco
RF3	SI	ALTA	SI	Sistema	Test gestione caratteristiche carta	Senza questo requisito non è possibile analizzare le carte
RF4	SI	ALTA	SI	Giocatore	Test confronto mazzi	Non sarebbe possibile confrontare i mazzi
RF5	SI	ALTA	SI	Sistema	Test algoritmo percentuale	Funzione principale del software
RF6	SI	ALTA	SI	Giocatore	Test visualizzazione risultati	L'utente non vedrebbe il risultato

2.3 Requisiti informativi

Il sistema gestisce diverse tipologie di dati.

Dati relativi alle carte:

• nome della carta
• costo in elisir
• tipo
• presenza evoluzione
• presenza eroe.

Dati relativi al mazzo:

• elenco delle carte
• numero totale di carte
• costo medio di elisir.

Dati relativi all'utente:

• nome utente
• eventuali informazioni di utilizzo del sistema.

3. REQUISITI NON FUNZIONALI E VINCOLI DEL SISTEMA

3.1 Interfaccia utente, fattori umani e ambiente fisico

L'interfaccia del sistema deve essere semplice e intuitiva.
Nella versione grafica del software l'utente potrà:

• selezionare le carte tramite menu a tendina
• visualizzare i due mazzi affiancati
• premere un pulsante Calcola per avviare l'analisi.

Il risultato deve essere facilmente leggibile.

3.2 Documentazione

Il sistema deve essere accompagnato da documentazione tecnica che includa:

• descrizione delle classi
• diagrammi UML
• descrizione dei casi d'uso
• documentazione del codice.

3.3 Requisiti hardware e software

Software richiesto:

• Java Development Kit (JDK 8 o superiore)
• ambiente di sviluppo (Eclipse, IntelliJ o simili).

Hardware richiesto:

• qualsiasi computer in grado di eseguire la
JVM.

3.4 Gestione degli errori e delle condizioni al limite

Il sistema deve gestire diverse condizioni di errore, tra cui:

• inserimento di valori di elisir non validi
• inserimento di più di 8 carte in un mazzo
• tentativo di confronto tra mazzi incompleti.

In questi casi il sistema deve segnalare l'errore all'utente.

3.5 Interfaccia di sistema

Il sistema può interagire con:

• un database o file contenente le informazioni delle carte
• librerie Java per la gestione dell'interfaccia grafica.

3.6 Prestazioni e qualità

Il sistema deve garantire tempi di risposta rapidi.

Il calcolo della percentuale di vittoria è effettuato con complessità O(n) dove n rappresenta il numero di carte del mazzo.

3.7 Manutenibilità

Il sistema è progettato seguendo il paradigma Object Oriented.

Questo permette di:

• aggiungere nuove carte facilmente
• modificare la logica di confronto
• estendere il sistema con nuove funzionalità.

4. MODELLO DEL SISTEMA

4.1 Elenco dei casi d'uso

UC1 - Creazione del mazzo
UC2 - Inserimento delle carte
UC3 - Confronto tra due mazzi
UC4 - Visualizzazione della percentuale di vittoria

4.2 Diagramma dei casi d'uso

Attore principale: Giocatore

Casi d'uso:
• inserire carte
• creare mazzo
• confrontare mazzi
• visualizzare risultati.

5. ANALISI DEL CASO D'USO "CALCOLO MATCHUP"

5.1 Scenario

Un giocatore vuole sapere quali sono le probabilità di vittoria del proprio mazzo contro un mazzo avversario.

Utilizzando il software inserisce le carte dei due mazzi e avvia l'analisi.

5.2 Nome e descrizione

Calcolo Matchup
Il sistema confronta due mazzi e calcola una percentuale di vittoria stimata.

5.3 Attori

Giocatore.

5.4 Pre-condizioni

• l'utente ha inserito il proprio mazzo
• l'utente ha inserito il mazzo avversario.

5.5 Flusso principale degli eventi

1. L'utente inserisce le carte del proprio mazzo.
2. L'utente inserisce le carte del mazzo avversario.
3. Il sistema verifica che i mazzi siano completi.
4. Il sistema calcola il costo medio di elisir.
5. Il sistema analizza le carte evoluzione ed eroe.
6. Il sistema calcola la percentuale di vittoria.
7. Il risultato viene mostrato all'utente.

5.6 Flussi alternativi

Se il mazzo non contiene 8 carte il sistema richiede di completare l'inserimento.

5.7 Condizioni di errore

Se uno dei mazzi è vuoto il sistema mostra un messaggio di errore.

5.8 Post-condizioni

Il sistema ha calcolato e mostrato la percentuale di vittoria.

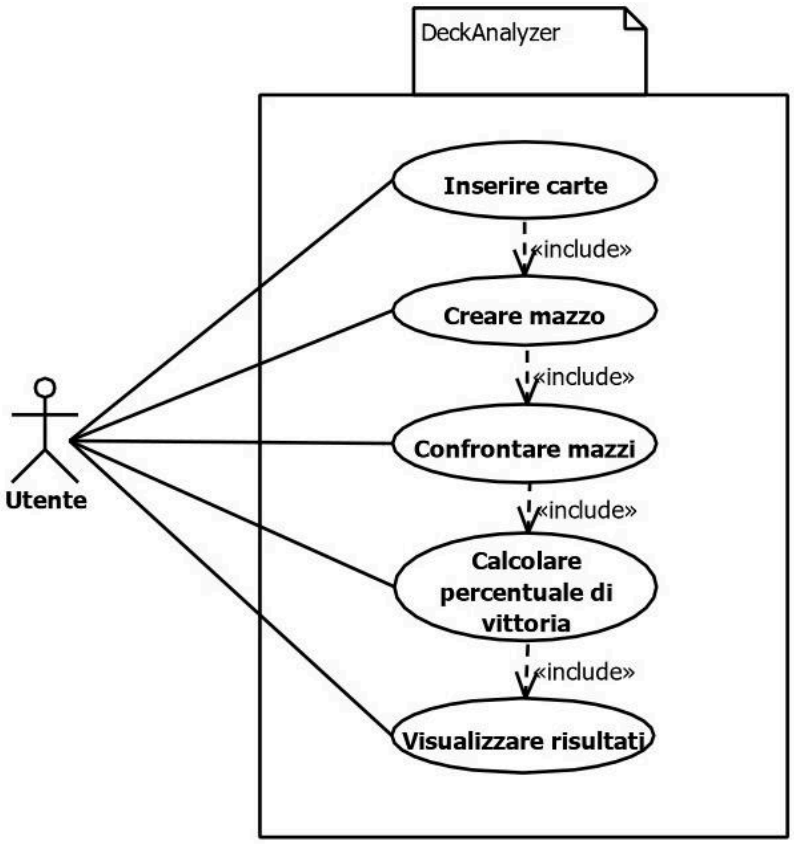
5.9 Priorità e frequenza d'uso

Questo è il caso d'uso principale del sistema.

5.10 Requisiti speciali

Il calcolo della percentuale deve essere rapido e preciso.

5.11 Diagramma dei casi d'uso



5.12 Diagramma delle classi

Classi principali del sistema:

Carta

Attributi:
• nome
• elisir
• tipo
• evO
• hero

Metodi:
• getter e setter
• inserisciDati()

Deck

Attributi:
• carte
• numeroCarte

Metodi:
• aggiuntaCarta()
• confrontaDeck()
• calcolaElisirMedio()

Main
Gestisce l'esecuzione del programma.

5.13 Diagramma di attività

Flusso:

Inizio
↓
Inserimento carte
↓
Controllo mazzo completo
↓
Calcolo statistiche
↓
Calcolo percentuale vittoria
↓
Visualizzazione risultato
↓
Fine

5.14 Diagramma di sequenza

Oggetti coinvolti:

Utente → Main → Deck → Carta

Sequenza:

1. Utente avvia il programma
2. Main crea i mazzi
3. Main aggiunge le carte
4. Main richiama confrontaDeck()
5. Deck restituisce percentuale
6. Main mostra il risultato

5.15 Diagramma di stato

Stati del sistema:

- Avvio
- Inserimento carte
- Mazzo completo
- Calcolo matchup
- Visualizzazione risultato
- Fine

6. APPENDICE

6.1 Glossario

Deck
Insieme di 8 carte utilizzate in una partita.

Matchup
Confronto tra due mazzi.

Elisir
Risorsa utilizzata per giocare le carte.

6.2 Documentazione

La documentazione del sistema comprende:

• codice sorgente
• diagrammi UML
• documento dei requisiti.

6.3 Riferimenti

• codice sorgente del progetto
• Modello documentazione fornito dalla professoressa Chiara C.
• Sito web DeckAI.